



目录

CONTENTS

1

机器学习简介

2

机器学习相关基础知识

3

K近邻算法 (KNN)

4

线性回归法

5

主成分分析法 (PCA)

6

逻辑回归法

7

支持向量机算法 (SVM)

8

决策树算法

9

集成学习和随机森林

1

逻辑回归

监督学习的最主要类型

标签离散

✓ 分类 (Classification)

- ✓ 身高1.85m , 体重100kg的男人穿什么尺码的T恤 ?
- ✓ 根据肿瘤的体积、患者的年龄来判断良性或恶性 ?
- ✓ 根据用户的年龄、职业、存款数量来判断信用卡是否会违约 ?

输入变量可以是离散的, 也可以是连续的。

1

逻辑回归

假设现在有一些数据点，用一条直线对这些点进行拟合，这个拟合过程就称作回归。而逻辑回归是一种预测分析，解释因变量与一个或多个自变量之间的关系。与线性回归不同的是它的目标变量有几种类别，所以逻辑回归主要用于解决分类问题。

逻辑回归可以看作是回归和分类之间的桥梁，是线性回归算法的一种拓展，当想将线性回归应用到分类问题中该怎么办呢？

线性回归本身的输出是连续的，若将线性回归应用到分类问题，则需要将连续的值调整为离散的0和1。因此，条件是找到一个预测**分类函数**，将X对应的y映射到(0,1)之间。这就是**逻辑回归的核心思想**。

与线性回归相比，逻辑回归使用概率的方式，预测出属于某一分类的概率值。如果概率值超过50%，则属于类别1，否则属于类别0。

1

逻辑回归基本原理

对于机器学习来说，本质就是求出一个函数 f ，有一个样本 x 输入到函数 f 中，最终得到的值通常是 $\hat{y}$ 。对于逻辑回归来说，是要得到一个函数 f ，把新样本 x 放入到函数 f 中， $f(x)$ 会计算出一个值 $\hat{p} = f(x)$ ，会根据概率值 $\hat{p}$ 来对样本进行分类。以上过程描述可用表达式表示如下：

$$\hat{y} = f(x) = \begin{cases} 1, & \hat{p} \geq 0.5 \\ 0, & \hat{p} \leq 0.5 \end{cases}$$

逻辑回归既可以看作是一个回归算法，也可以看作是一个分类算法。如果不进行最后一步，根据 $\hat{p}$ 的值进行分类操作的话，此时就是线性回归算法，输出的结果是个连续的值；如果进行最后一步，此时就是逻辑回归算法，通过样本的特征来拟合计算出一个事件发生的概率。通常情况下，使用逻辑回归都是用来当作分类算法用。分类不仅可以解决二分类问题，也可以解决多分类问题。

1

逻辑回归基本原理

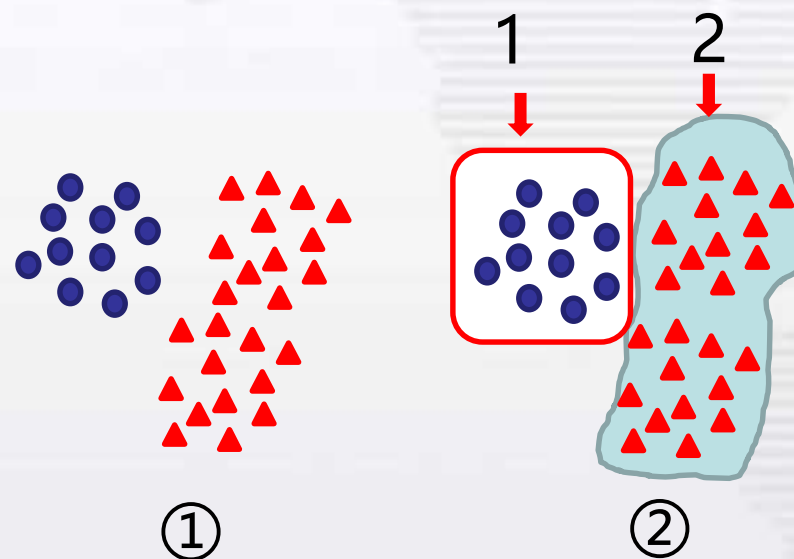
二分类

先从用蓝色圆形数据定义为类型1，

其余数据为类型2；

只需要分类1次

步骤：①->②



二分类

1

逻辑回归基本原理

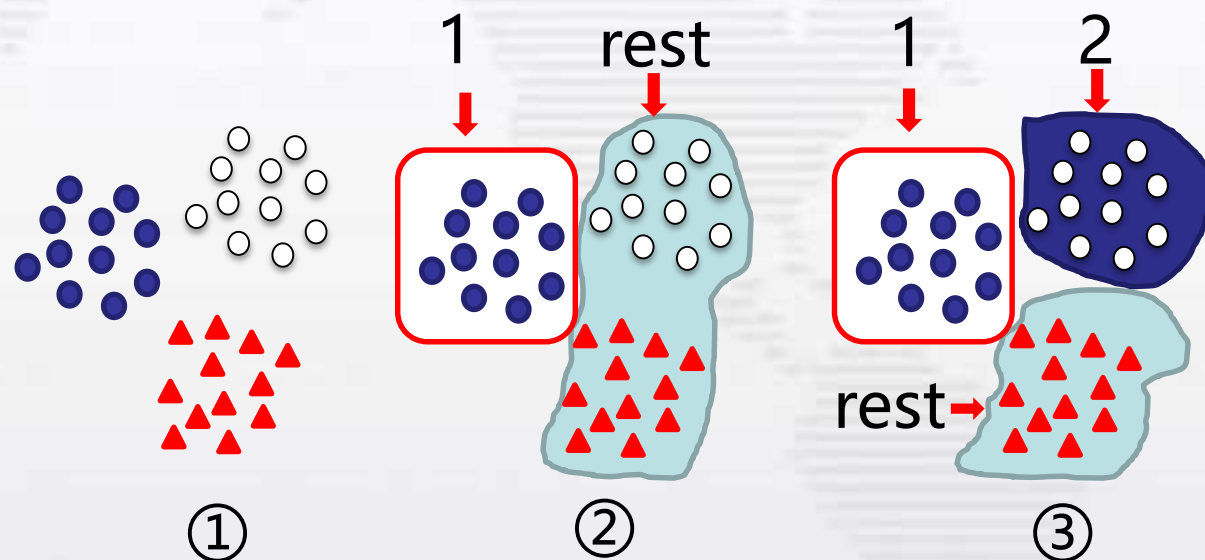
多分类

先定义其中一类为类型1（正类），
其余数据为负类（rest）；

接下来去掉类型1数据，剩余部分再
次进行二分类，分成类型2和负类；

如果有 n 类，那就需要分类 $n-1$ 次

步骤：①->②->③->.....



One-vs-All (One-vs-Rest)

一对多 (一对余)

2

逻辑回归计算步骤

按照逻辑回归的原理，求解过程可以分为以下三步：

- (1) 找一个合适的预测分类函数，用来预测输入数据的分类结果，一般表示为 f 函数，需要对数据有一定的了解和分析，然后确定函数的可能形式。
- (2) 构造一个损失函数，该函数表示预测输出值 $\hat{y}$ 与训练数据类别真实值 y 之间的偏差，一般是预测输出与实际类别的差，可对所有样本的偏差求 R^2 值等作为评价标准。
- (3) 找到损失函数的最小值，因为值越小表示预测函数越准确。损失函数的最小值采用梯度下降法进行求解。

2

预测分类函数

线性回归的函数 $h(x) = z = w^T x$, 范围是 $(-\infty, +\infty)$ 。

而分类预测结果需要得到 $[0,1]$ 的概率值。

在二分类模型中，事件的**几率odds**，即事件发生与事件不发生的概率之比为 $\frac{p}{1-p}$ ，称为

事件的发生比 (the odds of experiencing an event)

其中 p 为随机事件发生的概率， p 的范围为 $[0,1]$ 。

取对数得到： $\log \frac{p}{1-p}$ ，设 $\log \frac{p}{1-p} = w^T x = z$

求解得到： $p = \frac{1}{1+e^{-w^T x}} = \frac{1}{1+e^{-z}}$

2

预测分类函数

Sigmoid 函数

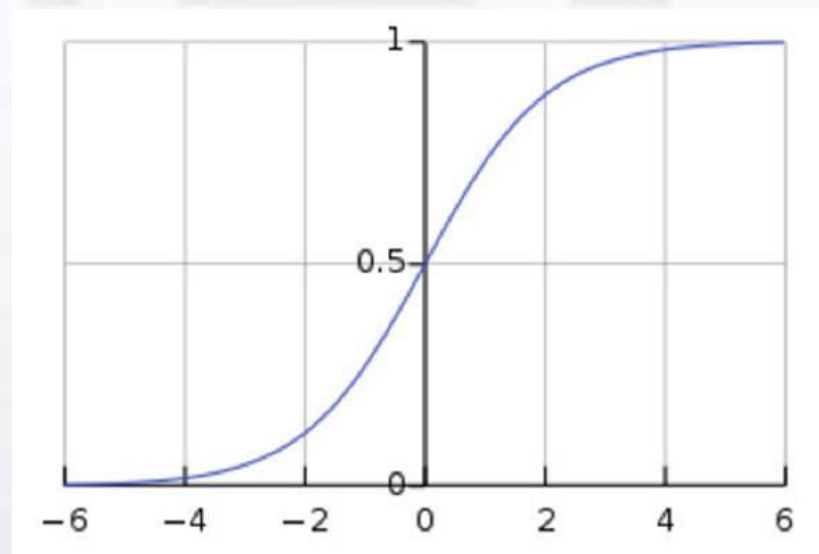
$\sigma(z)$ 代表一个常用的逻辑函数 (logistic function) 为Sigmoid函数，可以看出，Sigmoid函数连续，光滑，严格单调，以(0,0.5)为中心。当x趋近负无穷时，y趋近于0；当x趋近于正无穷时，y趋近于1。Sigmoid函数的值域范围限制在(0,1)之间，我们知道(0,1)与概率值的范围是相对应的，这样Sigmoid函数就能与一个概率分布联系起来了。

$$\text{则：}\sigma(z) = g(z) = \frac{1}{1+e^{-z}} \quad z = w^T x + b$$

合起来，得到逻辑回归模型的假设函数：

$$L(\hat{y}, y) = -y\log(\hat{y}) - (1 - y)\log(1 - \hat{y})$$

注意：若表达式 $h(x) = z = w_0 + w_1x_1 + w_2x_2 + \dots + w_nx_n + b = w^T x + b$ ，则b可以融入到 w_0 ，即： $z=w^T x$



当 $\sigma(z) \geq 0.5$ 时，预测 $y=1$

当 $\sigma(z) < 0.5$ 时，预测 $y=0$

2

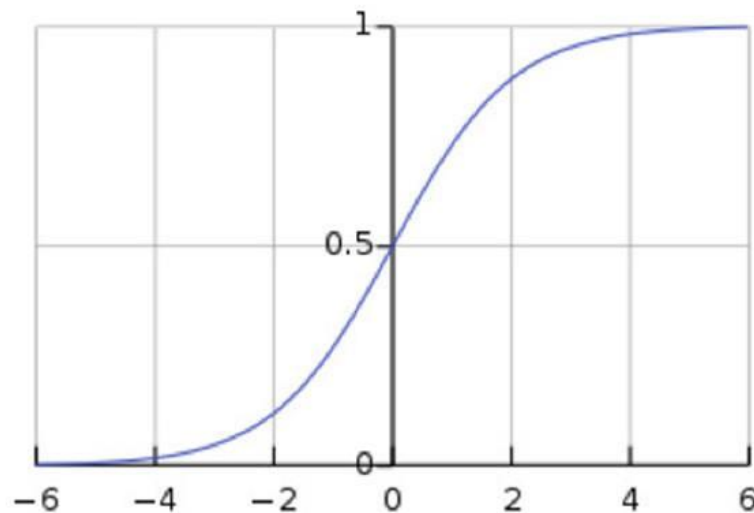
逻辑回归损失函数

逻辑回归函数： $g(z) = \frac{1}{1+e^{-z}}$

首先对逻辑回归函数求导（后面对loss求偏导会用到）

$$\begin{aligned} g'(z) &= \frac{\partial}{\partial z} \frac{1}{1+e^{-z}} = -\frac{1}{(1+e^{-z})^2} * (1+e^{-z})' \\ &= \frac{1}{(1+e^{-z})^2} (e^{-z})' \\ &= \frac{1}{1+e^{-z}} * \frac{e^{-z}}{1+e^{-z}} = \frac{1}{1+e^{-z}} * \left(\frac{1+e^{-z}}{1+e^{-z}} - \frac{1}{1+e^{-z}} \right) \\ &= \frac{1}{1+e^{-z}} * \left(1 - \frac{1}{1+e^{-z}} \right) \\ &= g(z)(1-g(z)) \end{aligned}$$

$$(1+e^{-z})' = (e^{-z})' = (e^{-z})(-z)' = (e^{-z})(-1)$$



$g(z)$

3

逻辑回归损失函数

在逻辑回归算法中有一个很重要的问题就是如何求出这个 w 。在线性回归中 X_b 点乘 w 之后得到的就是预测值，让预测值减去真实值的差来度量估计的好坏，用这个差的平方和作为损失函数，求出让损失函数最小的 w 即可。那么同理，对于逻辑回归也要这么做，整体的方向是一样的，只不过对于逻辑回归定义它的损失函数相对比较困难。

逻辑回归和线性回归最大的区别就在于逻辑回归解决的是一个分类问题，估计出来的 p 是一个概率，来决定我们估计的这个 y 到底是1还是0。它分成了两类，相应的损失函数也可以分为两类。

3

逻辑回归损失函数

假设一个二分类模型：

$$p(y = 1|x; w) = h(x)$$

$$p(y = 0|x; w) = 1 - h(x)$$

y 为某一条样本的预测
值,取值范围为0或者1.

则：

$$p(y|x; w) = (h(x))^y (1 - h(x))^{1-y}$$

逻辑回归模型的假设是： $h(x) = g(w^T x) = g(z)$

其中 $z = w^T x$

3

逻辑回归损失函数

此时想要找到一组 w ,使函数 $g(z) = \frac{1}{1+e^{-z}}$ 其中 $z = w^T x$ 正确的概率最大, 而前面的推理过程中已经得到每个单条样本预测正确概率的公式: $p(y|x; w) = (h(x))^y (1 - h(x))^{1-y}$

若想让预测出的结果全部正确的概率最大, 根据最大似然估计, 就是所有样本预测正确的概率相乘得到的 P (总体正确)最大, 此时让 $h(x) = g(z) = \frac{1}{1+e^{-z}}$, 数学表达形式如下:

$$L(\theta) = \prod_{i=1}^m (h_{\theta}(x^{(i)}))^{y^{(i)}} (1 - h_{\theta}(x^{(i)}))^{1-y^{(i)}}$$

<http://blog.csdn.net/dc>

3

逻辑回归损失函数

上述公式最大时公式中W的值就是最好的W，下面对公式进行求解。我们知道一个连乘的函数是不好计算的，可以通过两边同时取log的形式让其变成连加形式。

$$\begin{aligned} l(\theta) &= \log L(\theta) \\ &= \sum_{i=1}^m \left(y^{(i)} \log h_{\theta}(x^{(i)}) + (1 - y^{(i)}) \log(1 - h_{\theta}(x^{(i)})) \right) \end{aligned}$$

得到的这个函数值越大，证明得到的W就越好。因为在函数最优化的时候习惯让一个函数越小越好，所以在前边加一个负号，得到公式如下：

$$J_{\log}(w) = \sum_{i=1}^m -y_i \log(p(x_i; w)) - (1 - y_i) \log(1 - p(x_i; w))$$

这个函数就是逻辑回归(logistics regression)的损失函数，也称为交叉熵损失函数。

3

逻辑回归损失函数

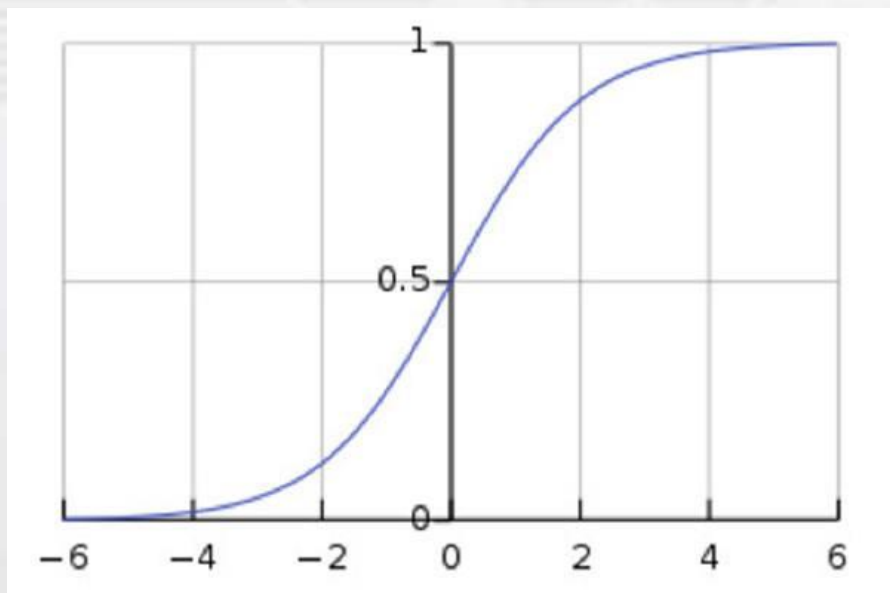
交叉熵损失函数

$$J_{\log}(w) = \sum_{i=1}^m -y_i \log(p(x_i; w)) - (1 - y_i) \log(1 - p(x_i; w))$$

$$L(\hat{y}, y) = -y \log(\hat{y}) - (1 - y) \log(1 - \hat{y})$$

$\hat{y}$ 表示预测值 $h(x)$

y 表示真实值



y 为某一条样本的预测值，取值范围为0或者1；

(1) 如果 y 等于1，表示样本的真实类别是1，此时估计的 $\hat{y}$ 越小，即预测样本的类别趋近于0，损失函数 $J(w)$ 越大。

(2) 如果 y 等于0，表示样本的真实类别是0，此时估计的 $\hat{y}$ 越大，即预测样本的类别趋近于1，估计错误，损失函数 $J(w)$ 越大。

3

逻辑回归损失函数

为了衡量算法在全部训练样本上的表现如何，需要定义一个算法的代价函数，算法的代价函数是对 m 个样本的损失函数求和然后除以 m ：

代价函数

$$J(w) = \frac{1}{m} \sum_{i=1}^m L(\hat{y}^{(i)}, y^{(i)}) = \frac{1}{m} \sum_{i=1}^m \left(-y^{(i)} \log \hat{y}^{(i)} - (1 - y^{(i)}) \log(1 - \hat{y}^{(i)}) \right)$$

逻辑回归的损失函数不像线性回归的损失函数一样非常容易地利用最小二乘法求出一个正规方程解，实际上对于这个式子来说，不能列出一个公式把 x 和 y 的值直接套进公式里得到 w 。此时需要用到的方法是梯度下降法。

4

逻辑回归求解

梯度下降求解过程：

$$w_j := w_j - \alpha \frac{\partial J(w)}{\partial w}$$

其中 $-\alpha \frac{\partial J(w)}{\partial w}$ 来更新参数 w ，不断地迭代更新以此求得损失函数的局部/全局最优解。

表示的含义是利用梯度下降法来决定应该从哪个方向下降，每一步走多长。

$$J(w) = -\frac{1}{m} \sum_{i=1}^m (y^{(i)} \log(h(x^{(i)})) + (1 - y^{(i)}) \log(1 - h(x^{(i)})))$$

$$\frac{\partial}{\partial w_j} J(w) = \frac{1}{m} \sum_{i=1}^m (h(x^{(i)}) - y^{(i)}) x_j^{(i)}$$

$$\text{则：} w_j := w_j - \alpha \frac{1}{m} \sum_{i=1}^m (h(x^{(i)}) - y^{(i)}) x_j^{(i)}$$

4

逻辑回归求解

求解过程： $\frac{\partial}{\partial w_j} J(w) = \frac{1}{m} \sum_{i=1}^m (h(x^{(i)}) - y^{(i)}) x_j^{(i)}$ 的推导过程：

$$J(w) = -\frac{1}{m} \sum_{i=1}^m (y^{(i)} \log(h(x^{(i)})) + (1 - y^{(i)}) \log(1 - h(x^{(i)})))$$



$$\begin{aligned} & y^{(i)} \log(h(x^{(i)})) + (1 - y^{(i)}) \log(1 - h(x^{(i)})) \\ &= y^{(i)} \log\left(\frac{1}{1 + e^{-w^T x^{(i)}}}\right) + (1 - y^{(i)}) \log\left(1 - \frac{1}{1 + e^{-w^T x^{(i)}}}\right) \\ &= -y^{(i)} \log(1 + e^{-w^T x^{(i)}}) - (1 - y^{(i)}) \log(1 + e^{w^T x^{(i)}}) \end{aligned}$$

4

逻辑回归求解

求解过程： $\frac{\partial}{\partial w_j} J(w) = \frac{1}{m} \sum_{i=1}^m (h(x^{(i)}) - y^{(i)}) x_j^{(i)}$ 的推导过程：

$$\begin{aligned} \frac{\partial}{\partial w_j} J(w) &= \frac{\partial}{\partial w_j} \left(-\frac{1}{m} \sum_{i=1}^m \left(-y^{(i)} \log(1 + e^{-w^T x^{(i)}}) - (1 - y^{(i)}) \log(1 + e^{w^T x^{(i)}}) \right) \right) \\ &= -\frac{1}{m} \sum_{i=1}^m \left(-y^{(i)} \frac{-x_j^{(i)} e^{-w^T x^{(i)}}}{1 + e^{-w^T x^{(i)}}} - (1 - y^{(i)}) \frac{x_j^{(i)} e^{w^T x^{(i)}}}{1 + e^{w^T x^{(i)}}} \right) \\ &= -\frac{1}{m} \sum_{i=1}^m (y^{(i)} - h(x^{(i)})) x_j^{(i)} \\ &= \frac{1}{m} \sum_{i=1}^m (h(x^{(i)}) - y^{(i)}) x_j^{(i)} \end{aligned}$$

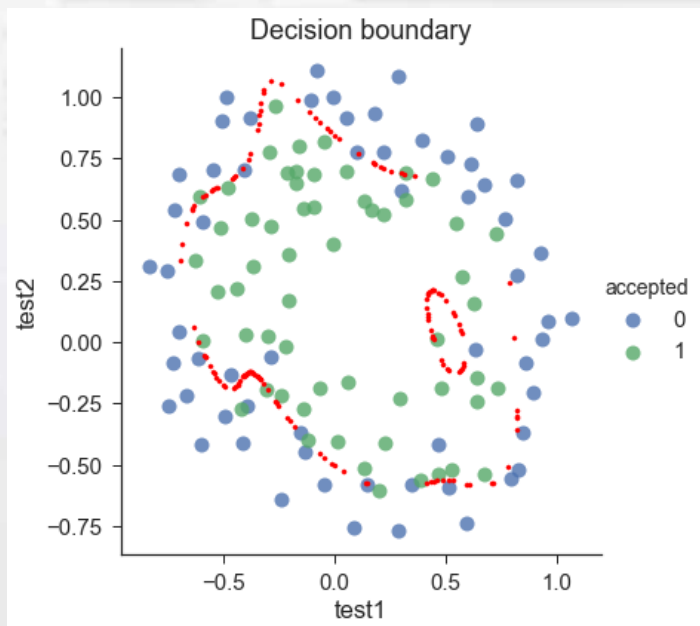
4

逻辑回归求解

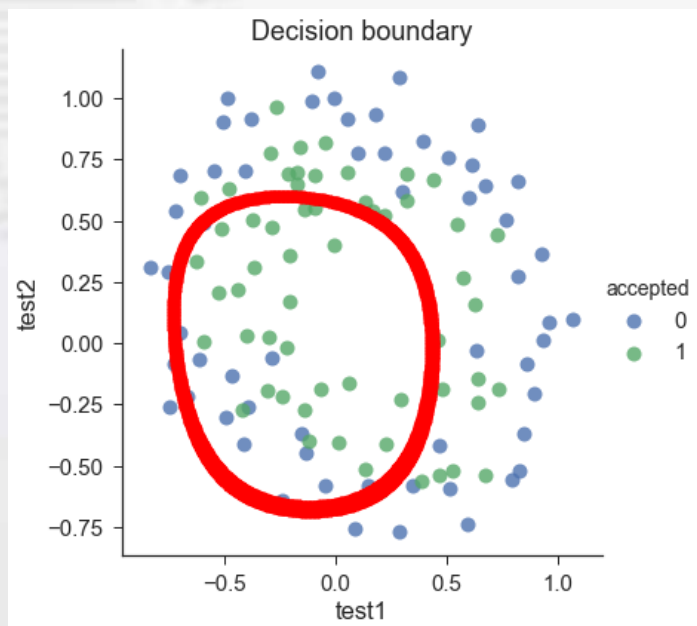
正则化：目的是为了防过拟合

$$J(w) = \frac{1}{m} \sum_{i=1}^m \left[-y^{(i)} \log(h(x^{(i)})) - (1 - y^{(i)}) \log(1 - h(x^{(i)})) \right] + \underbrace{\frac{\lambda}{2m} \sum_{j=1}^n w_j^2}_{\text{正则化项}}$$

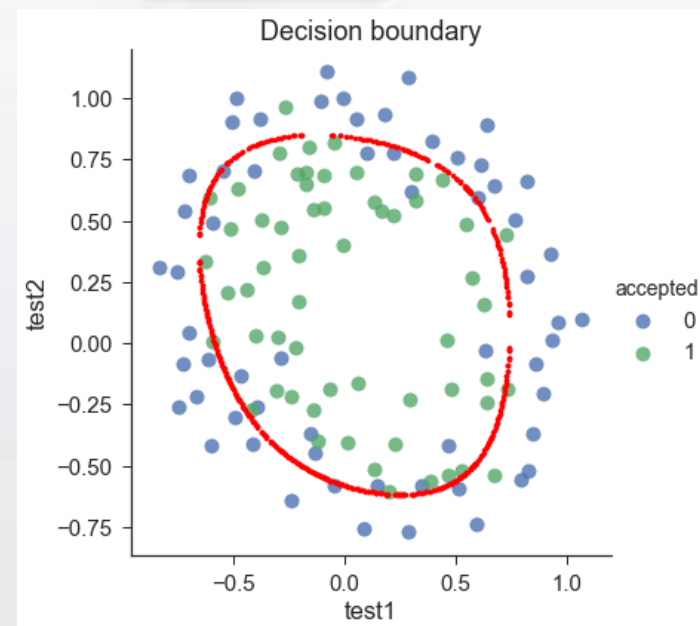
当 λ 的值开始上升时，
降低了方差。



没有正则化，过拟合



正则化过度，欠拟合



适当的正则化

5

逻辑回归代码实现

Sigmoid 函数

$$\sigma(z) = g(z) = \frac{1}{1+e^{-z}}$$

```
def sigmoid(z):  
    return 1 / (1 + np.exp(-z))
```

代价函数

$$J(w) = -\frac{1}{m} \sum_{i=1}^m (y^{(i)} \log(h(x^{(i)})) + (1 - y^{(i)}) \log(1 - h(x^{(i)})))$$

```
def cost(w, X, y):  
    w = np.matrix(w)  
    X = np.matrix(X)  
    y = np.matrix(y)  
    first = np.multiply(-y, np.log(sigmoid(X * w.T)))  
    second = np.multiply((1 - y), np.log(1 - sigmoid(X * w.T)))  
    return np.sum(first - second) / (len(X))
```

5

逻辑回归代码实现

正则化

$$J(w) = \frac{1}{m} \sum_{i=1}^m \left[-y^{(i)} \log(h(x^{(i)})) - (1 - y^{(i)}) \log(1 - h(x^{(i)})) \right] + \frac{\lambda}{2m} \sum_{j=1}^n w_j^2$$

```
def costReg(w, X, y, LearningRate):  
    w = np.matrix(w)  
    X = np.matrix(X)  
    y = np.matrix(y)  
    first = np.multiply(-y, np.log(sigmoid(X * w.T)))  
    second = np.multiply((1 - y), np.log(1 - sigmoid(X * w.T)))  
    reg = (learningRate /  
           (2 * len(X))) * np.sum(np.power(w[:, 1:w.shape[1]], 2))  
    return np.sum(first - second) / len(X) + reg
```



谢谢